

1. Q1 FedAvg Implementation

The codes for Q1, are included in the corresponding folder (Q1).

The following algorithm was applied:

Algorithm 1 FederatedAveraging. The K clients are indexed by k ; B is the local minibatch size, E is the number of local epochs, and η is the learning rate.

Server executes:

```
initialize  $w_0$ 
for each round  $t = 1, 2, \dots$  do
   $m \leftarrow \max(C \cdot K, 1)$ 
   $S_t \leftarrow$  (random set of  $m$  clients)
  for each client  $k \in S_t$  in parallel do
     $w_{t+1}^k \leftarrow \text{ClientUpdate}(k, w_t)$ 
   $m_t \leftarrow \sum_{k \in S_t} n_k$ 
   $w_{t+1} \leftarrow \sum_{k \in S_t} \frac{n_k}{m_t} w_{t+1}^k$ 
```

```
ClientUpdate( $k, w$ ): // Run on client  $k$ 
 $\mathcal{B} \leftarrow$  (split  $\mathcal{P}_k$  into batches of size  $B$ )
for each local epoch  $i$  from 1 to  $E$  do
  for batch  $b \in \mathcal{B}$  do
     $w \leftarrow w - \eta \nabla \ell(w; b)$ 
  return  $w$  to server
```

Note that some of the functions such as batching, and running training during each epoch, and updating the weights based on each batch are embedded functions in built-in tensorflow functions that we are allowed to use. Please refer to the following documentation: https://www.tensorflow.org/api_docs/python/tf/keras/Model#fit

2. Reasonable description of data class distribution as required supported by histogram plots.

Figures 1 and 2 show the data distribution of the classes of the 100 clients. In figure 2, we can also see class 24 has the highest frequency among all the 100 clients. And the average frequency among all classes for the overall dataset of all clients is 513.13 (divided by 100, that's around 5.13). And the standard deviation of the frequency among all classes is close to the average.

Figures 3,4,5,6,7 show the histogram of the data distribution of clients #1, 11, 43, 65, 82. Looking at these 5 figures along figure 2, we notice that the histogram have a similar pattern, although the exact distributions are slightly different. The similarities between the distributions of the data include:

- Only a few classes have significantly higher frequency than the rest of the classes, and these are almost the same for all the clients. Notice that some clients share the same mode class.
- Classes 1 to 10 have a medium frequency and are somewhat uniform in frequency.
- The rest of the classes have significantly lower frequency.
- The average frequency among all classes ranges between [4,6], same as the standard deviation of the frequency.

In conclusion we can notice that the distribution of classes between the clients is somewhat homogeneous and this shows that the data is i.i.d which is typically a condition for using FedAvg algorithm effectively.

Federated Average (FedAvg) is more effective when the data among clients is relatively identically distributed (IID) or, at the very least, similarly distributed. This means that the statistical properties of the data from different clients are comparable or have a resemblance.

While FedAvg can still function with non-IID data, a high degree of divergence in data distributions among clients might affect its performance. Federated Learning techniques often include strategies like weighted averaging, adaptive learning rates, or client selection mechanisms to handle such scenarios better.

However, in this homework, we are asked to randomly select clients, and we are asked to use weighted averaging, and using adaptive learning rates is optional.

When training the model, it was noticed that indeed achieving convergence was challenging, and on some clients, such as clients #3 and 47 (indices 2 and 46 in python), that maintained significantly lower validation accuracy as compared to other clients during communication rounds. To understand this variation, I also plotted the distributions of clients #3 and 47 in Figures 8 and 9, and I noticed that indeed these two clients have slightly more different distributions and even their mode classes and average frequency is slightly different than other clients. We can only notice this difference in client #82.

Another challenge in achieving good test accuracy is the fact that some clients have missing classes in their data, and a significantly smaller data for other classes. On the other hand, the test data includes all 62 classes as shown in Figure 10. This makes it difficult to achieve high accuracy without being able to have a smarter selection strategy of clients that is not allowed in this homework.

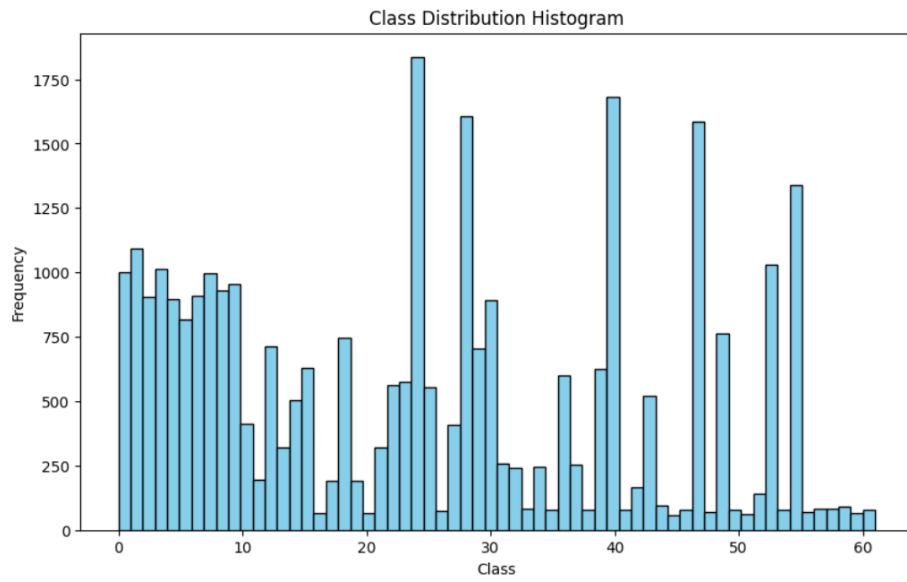


Figure 1: Data Distribution of all 100 clients

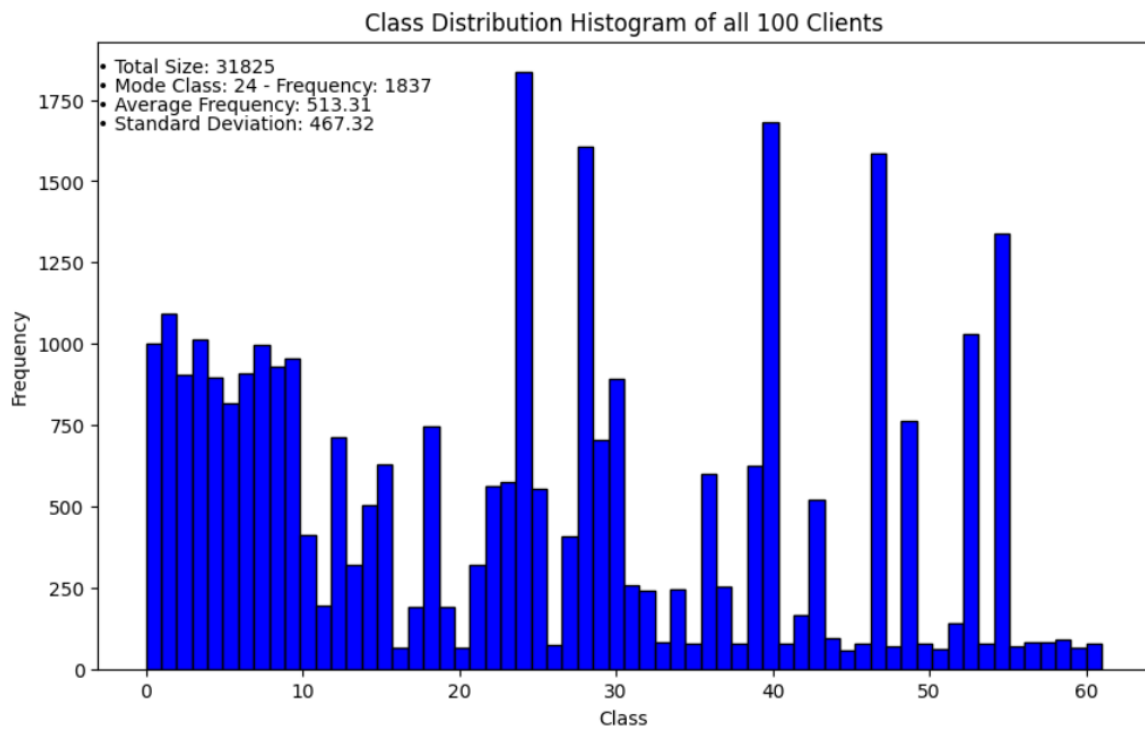


Figure 2: Data Distribution of all 100 clients with statistical information

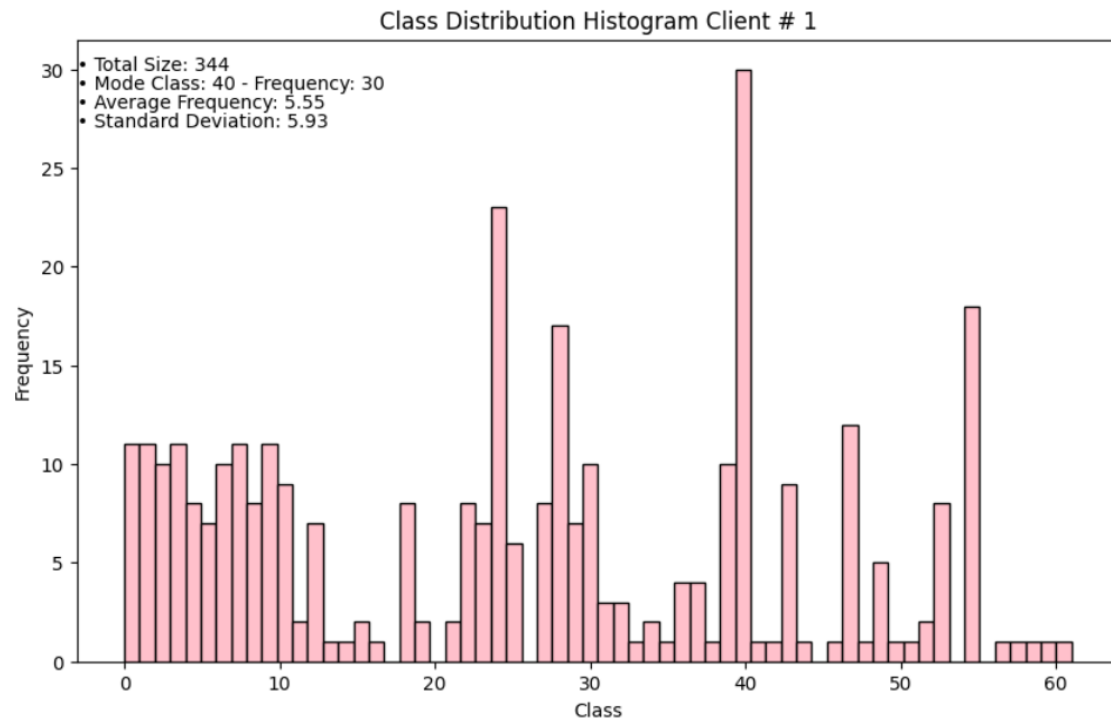


Figure 3: Labels Distribution for Client #1

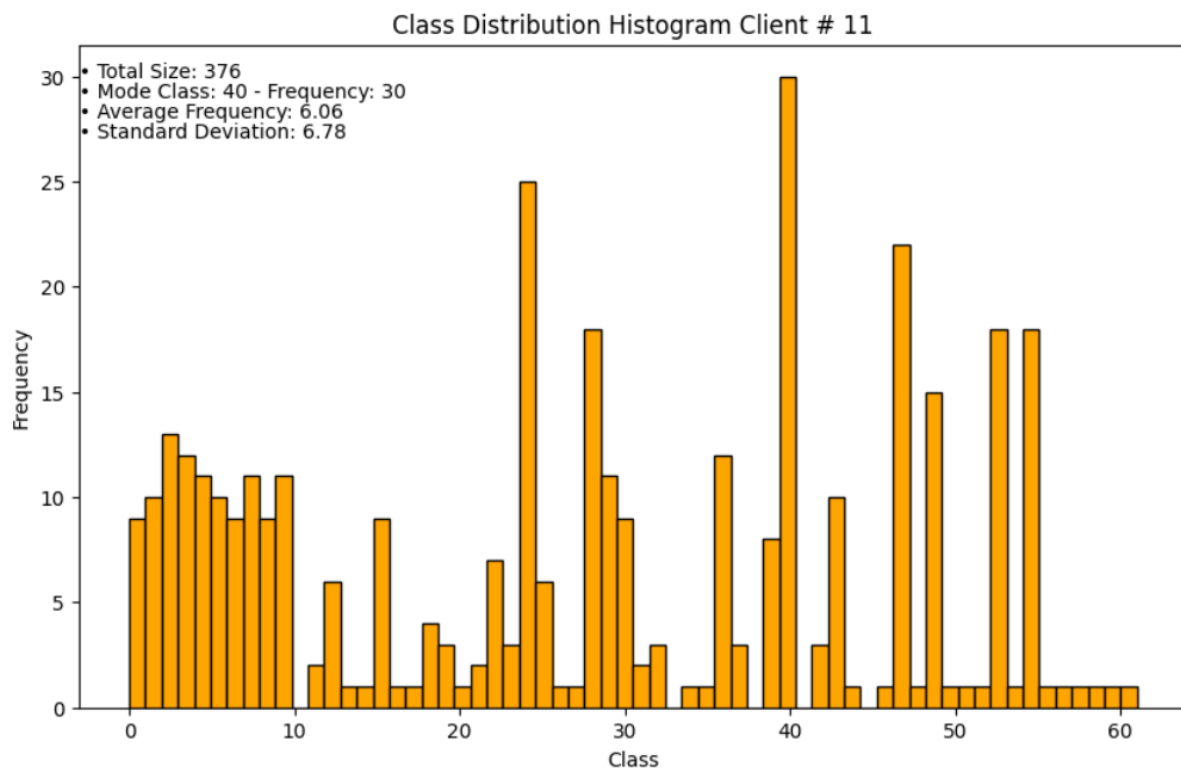


Figure 4: Labels Distribution of Client #11

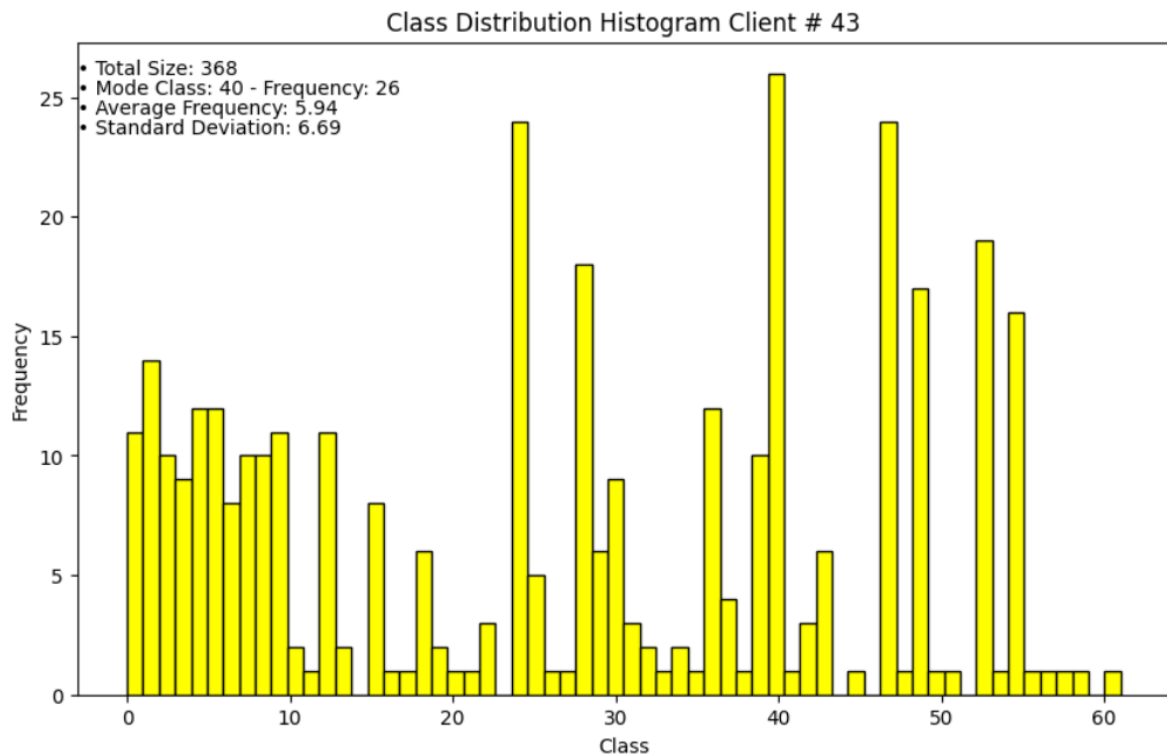


Figure 5: Labels Distribution for Client #43

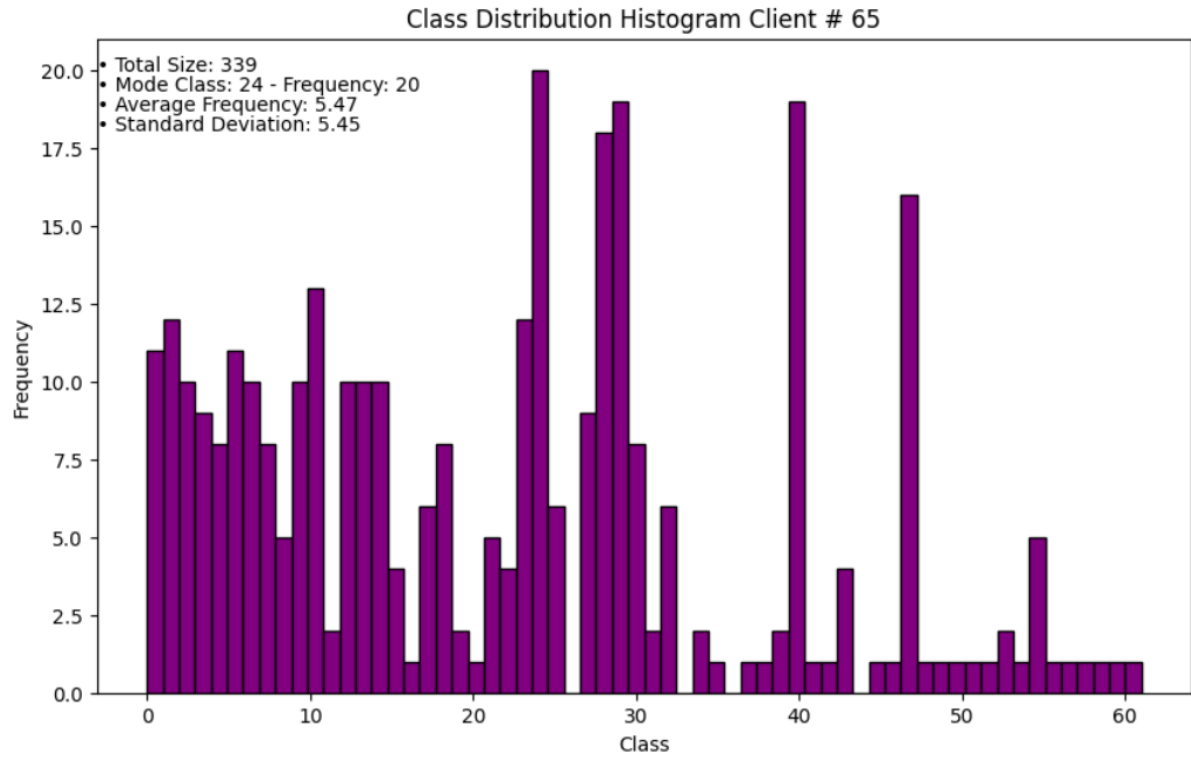


Figure 6: Labels Distribution of Client #65

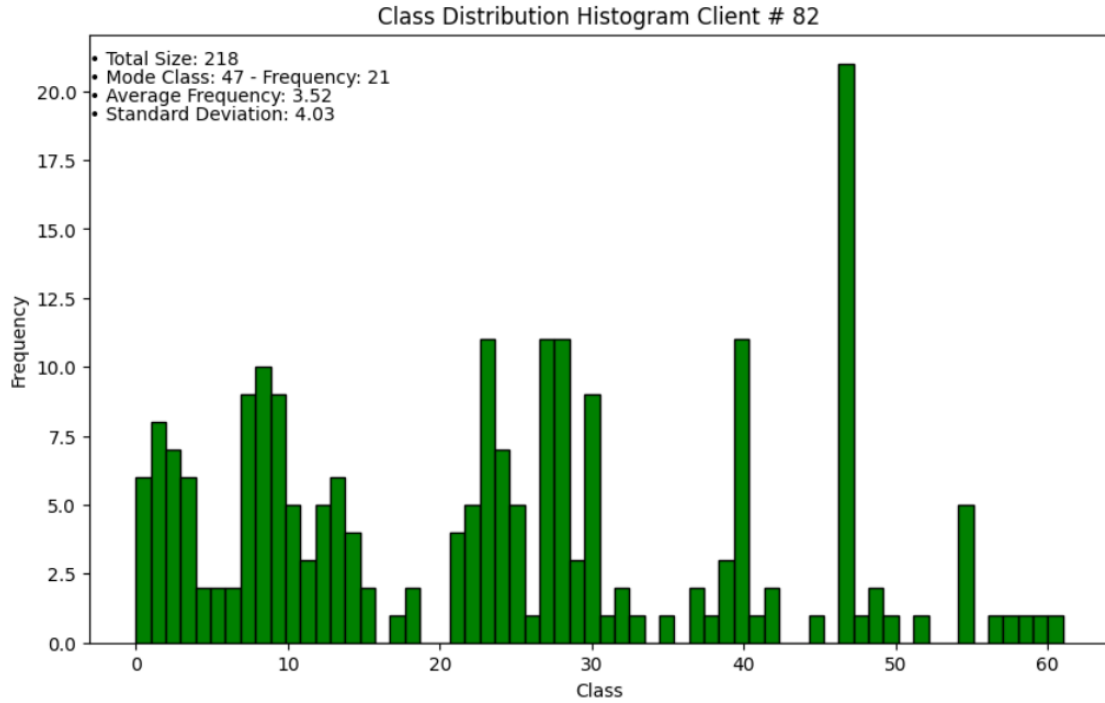


Figure 7: Labels Distribution of Client #82

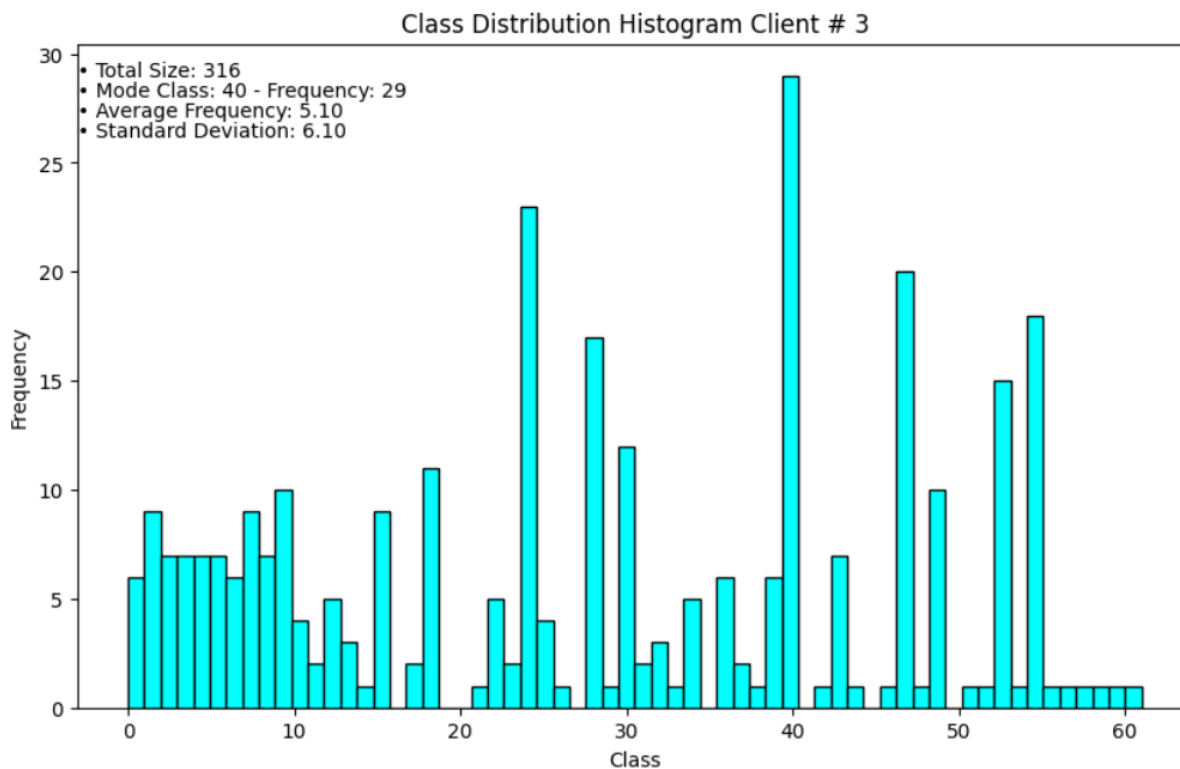


Figure 8: Labels Distribution of Client #3

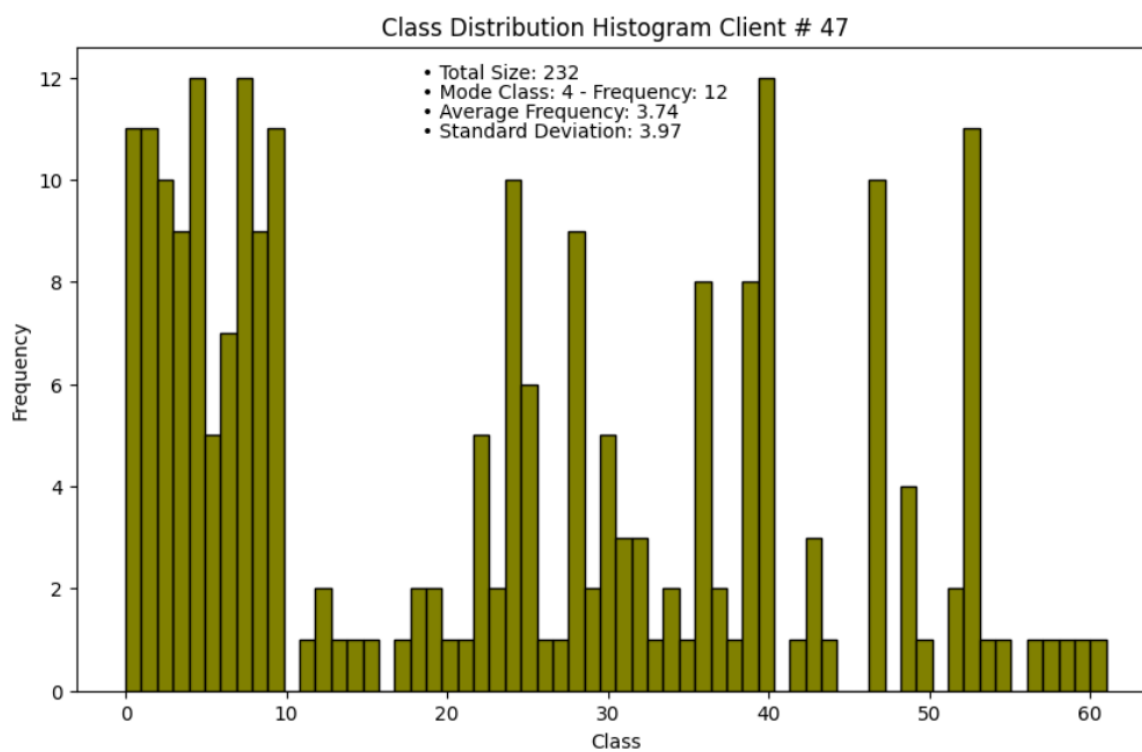


Figure 9: Labels Distribution of Client #47

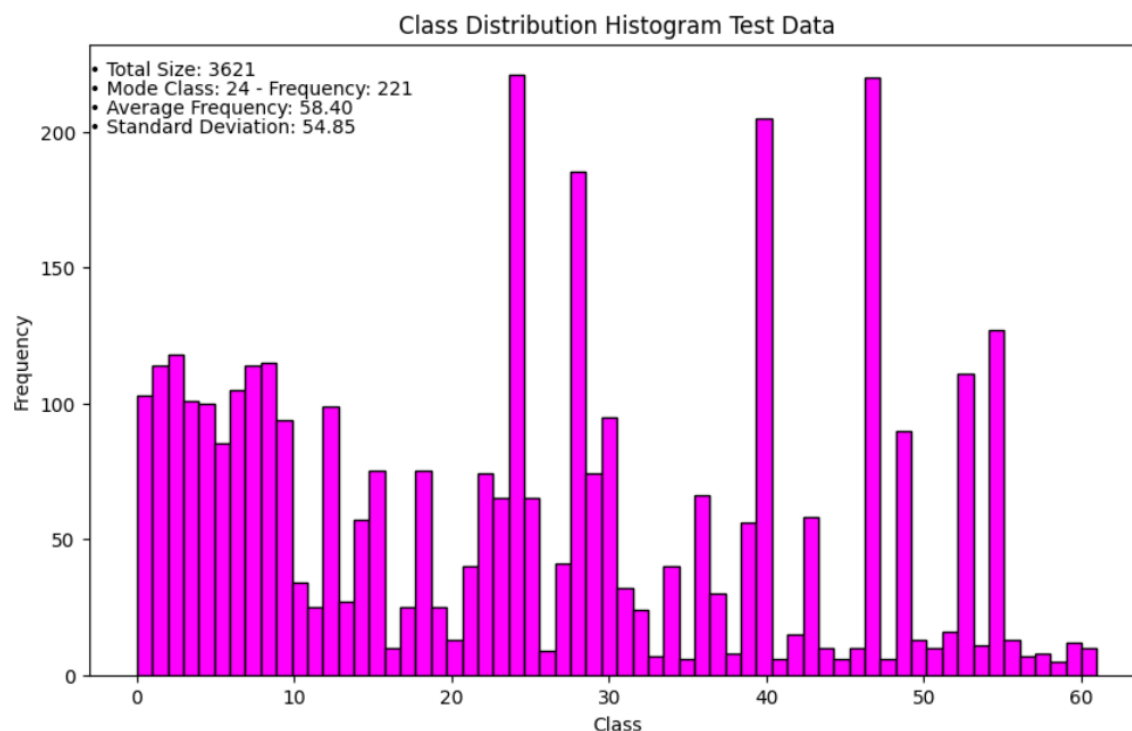


Figure 10: Label Distribution of Test Data

3. Q1 FedAvg Plots

The FedAvg algorithm was tested on a number of hyperparameters:

- Number of clients
- Number of epochs
- Batch size
- Learning rate
- Optimizer type
- Communication rounds number

The first two are tested in the next question. However, in general, when increasing epochs > 50, convergence is faster, and the starting training/validation accuracy is higher. While starting with lower epoch such as 10, 20, leads to smaller starting accuracy, and slower convergence.

The batch size determines the number of samples used in each iteration of training. Larger batch sizes can provide better hardware utilization and faster computation due to parallelism. However, excessively large batch sizes might not fit into memory or might lead to poor generalization. And I indeed ran into this problem when I used batch size equal to 64, for some runs. On the other hand, smaller batch sizes can help the model generalize better as they introduce more noise and prevent the model from getting stuck in local minima. However, smaller batch sizes can also be slower to converge, and I tested batch sizes that are

as low as 8. And in a federated learning setting specifically, the batch size can impact communication costs between the central server and clients. Larger batch sizes might lead to higher communication overhead even when using ray.

The learning rate controls the step size in updating model weights during training. A larger learning rate can lead to faster convergence as it takes larger steps in weight space. However, it might cause the model to overshoot the minimum and diverge. Smaller learning rates might lead to more stable training but can be slower to converge. And indeed that was the case when I used small learning rate as small as 0.01. For this reason, I used smaller learning rates. Adaptive learning rate methods, like Adam, RMSprop, or AdaGrad, adjust the learning rate dynamically for each parameter based on the historical gradients, helping avoid getting stuck in local minima.

I tested SGD and Adam optimizers. The former has more consistent learning but much slower, while the latter has faster learning but it fluctuates a bit more. However, in general SGD required longer to converge, so I used Adam.

Increasing communication rounds is not useful when the model has converged, and in some cases, it had led to a decrease in accuracy possibly due to catastrophic forgetting (for $T > 800$). So typically, I tested communication rounds between 100 and 800.

Models when data is shuffled were also tested, but no significant impact was found. Shuffling the data was still included. Normalizing the data by dividing pixels by 255, made convergence slower, to an extent that it was deemed unhelpful. Processing the data requires introducing additional layers in the neural network which wasn't permitted in this homework. The hyperparameters below balance between computation time (high epochs require high computation time so they're avoided), stable learning (thus using smaller epoch), and low memory overhead (smaller batch size). However, larger learning rates have showed low improvement in training, so instead I used a small learning rate. To avoid catastrophic forgetting that is common in neural networks, I used a learning scheduler.

I tested several runs, before I settled on the following hyperparameters:

- `batch_size = 16`
- `initial_learning_rate = 0.0001`
- `total_rounds = 1000`
- `C = 10`
- `E = 20`
- Adam optimizer
- Used learning scheduler.

The final aggregated validation accuracy of this model was around 0.702. The model of this run is in folder: “Main - C=10 E=20”.

The resulting training and validation accuracy are shown in Figure 11, and the resulting training and validation loss are shown in Figure 12:

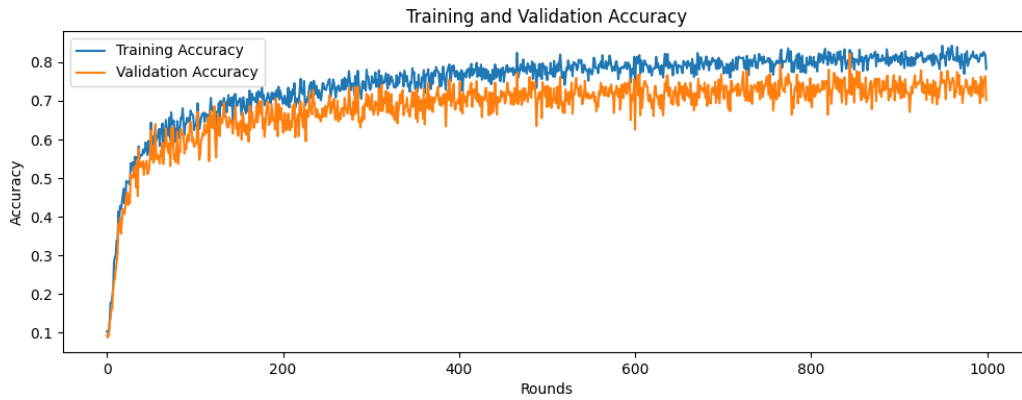


Figure 11: Training and Validation Accuracy of main Q1 run



Figure 12: Training and Validation Loss of main Q1 run

In both Figures 11 and 12 we notice that the model is learning and converges, and as expected there is some gap between the training and validation accuracy. However, the smaller the gap the better; which shows less overfitting. In contrast to some runs where training accuracy was higher towards the end (about 0.97), however the gap between the training and validation accuracy was higher, leading to a lower test accuracy.

The result of the prediction on the test data as compared to the actual test labels are summarized in Table 1:

Test Loss	1.5591191053390503
Test Accuracy	0.6915217041969299

These values look promising given that we were only allowed to use a simple neural network and only FedAVG.

4. Q1 FedAvg Test Accuracy

After training a model, the model was tested on the test data, refer to the code in this folder: Part 1\Q1\Main - C=10 E=20\ . The results are in the part above.

5. Q1 FedAvg Comparison

The number of epochs determines how many times the entire dataset is processed during training. Inadequate epochs might prevent the model from learning the underlying patterns in the data, leading to underfitting when the number of epochs is low. The model may not capture complex relationships present in the data. And indeed this happened when I used epoch=10. The model was stuck at low accuracy for too long (over 100 rounds). Excessive epochs might lead to overfitting, where the model learns not just the underlying patterns but also the noise in the data, resulting in poor generalization to unseen data. Using fewer Epochs reduces training time but might compromise model performance.

In federated learning, epoch numbers need to balance the communication cost with model convergence. Higher epoch counts might necessitate more communication rounds between the central server and clients, increasing the communication overhead.

And regarding C (number of clients per round), a larger number of clients contribute diverse local datasets, since in this dataset some clients have missing classes. So having more clients at each round increase the likelihood of training the model on all classes in each round. This diversity can potentially enhance the model's ability to generalize to different data distributions and improve overall model performance. On the other hand, it can increase the amount of communication needed between the central server and clients during each round of training. This can result in higher communication costs due to more frequent updates exchanged between clients and the central server. In case clients are non-i.i.d, this heterogeneity can pose challenges in aggregating models from clients with differing data distributions or varying local dataset sizes, impacting fairness in model updates across clients. This has been a problem for this dataset, slightly. In general, balancing the number of clients in FedAvg involves considering trade-offs between communication overhead, diversity, fairness, and system complexity. The optimal number of clients depends on various factors like the nature of the problem, the heterogeneity of the data, computational resources, and privacy concerns within the federated learning environment.

Comparisons at T=200

The model used for the 4 runs below include the following hyperparameters:

- Batch size = 32
- Learning rate = 0.001

$C=5, E=20$

The results of this run are below:

test loss: 2.043142318725586

test accuracy: 0.541563093662262

last train acc: 0.7258409554753286

last validation acc: 0.6592945968002961

[22]:

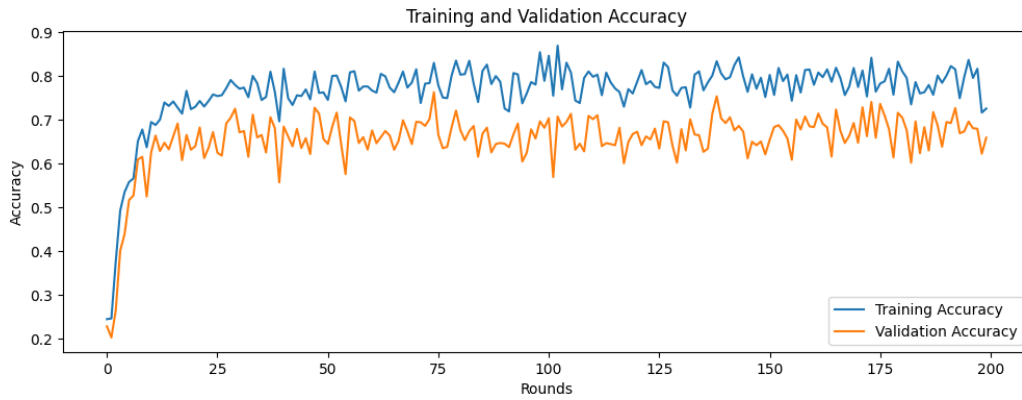


Figure 13: Aggregated accuracy of run with $C = 5, E=20$

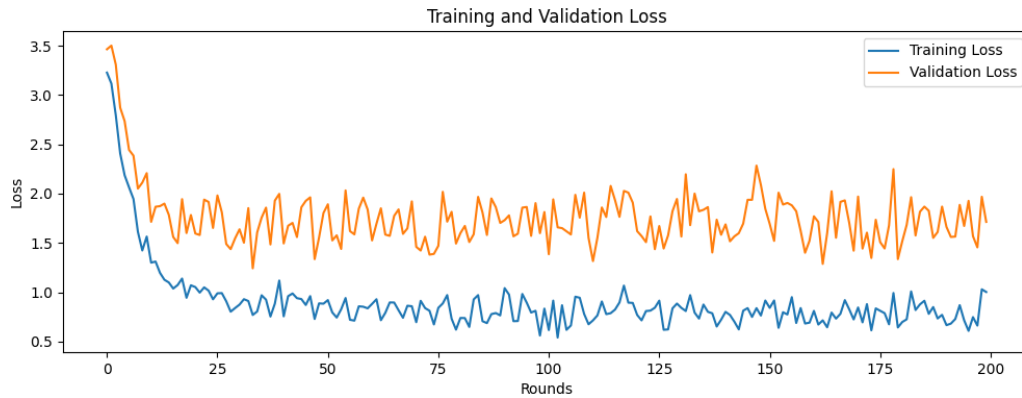


Figure 14: Aggregated loss of run with $C = 5, E=20$

$C=5, E=100$

The results of this run are below:

test loss: 2.9032373428344727

test accuracy: 0.5785694718360901

last train acc: 0.9656917563306445

last validation acc: 0.7216261926814719

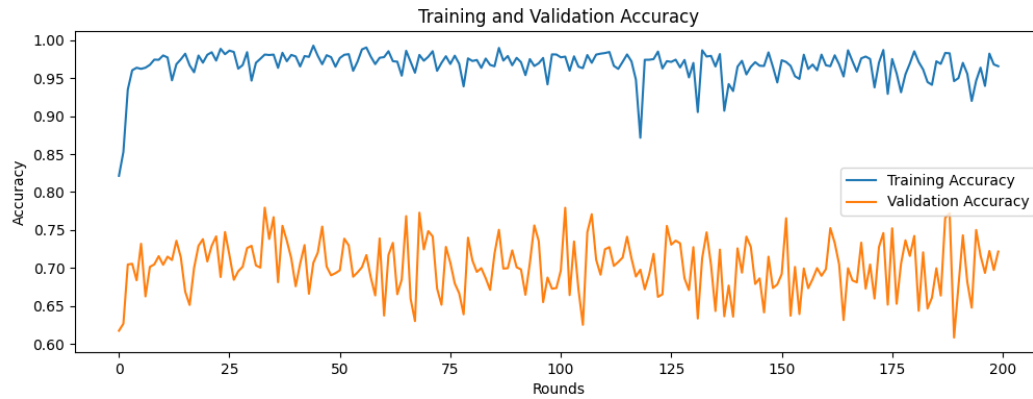


Figure 15: Aggregated accuracy of run with $C = 5$, $E=100$



Figure 16: Aggregated loss of run with $C = 5$, $E=100$

$C=8$, $E=20$

The results of this run are below:

last train acc: 0.8337148912151731

last validation acc: 0.712373536827449

test loss: 1.9015841484069824

test accuracy: 0.6023197770118713

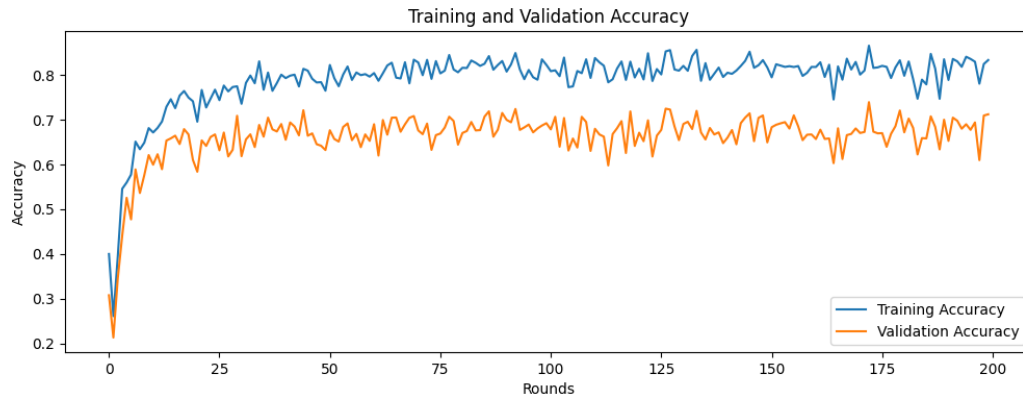


Figure 17: Aggregated accuracy of run with $C = 8$, $E=20$



Figure 18: Aggregated loss of run with $C = 8$, $E=20$

$C=8$, $E=100$

The results of this run are below:

train acc: 0.9680873942071277
validation acc: 0.7118296872481311
test loss: 2.693075180053711
test accuracy: 0.5766363143920898

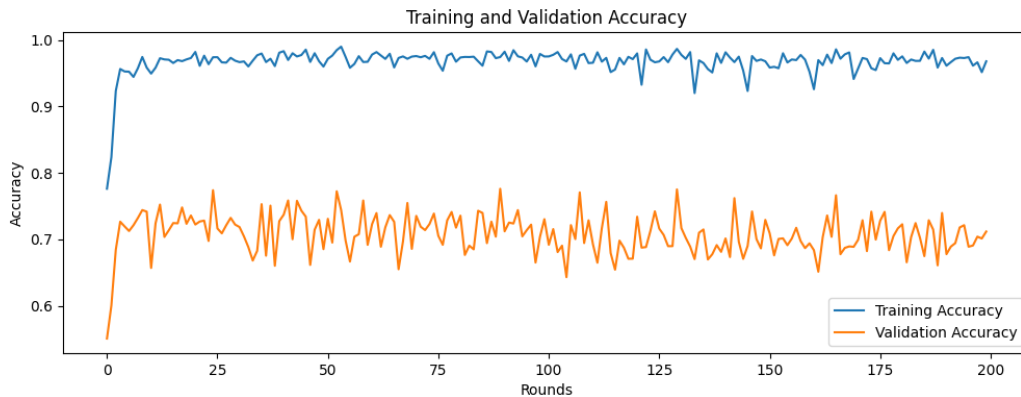


Figure 19: Aggregated accuracy of run with $C = 8$, $E=100$



Figure 20: Aggregated loss of run with $C = 8$, $E=100$

Looking at the plots above and values of each of the 4 runs we notice that:

- Using $E=100$ versus 20, leads to a higher gap between training and validation accuracy which is a sign of a complex model that has overfitted the data. Although both validation accuracies are almost the same, the training accuracy pace is way faster with epoch =100. So, using higher epoch could lead to less stable training, overfitting, and unreliable models.
- However, for $C=5$, increasing epochs to 100 improved both validation and test accuracy. This could mean that with conflicting data (because averaging over 5 leads could lead to slower convergence than averaging over 8 clients), higher epoch can improve the model and balance the conflicting weights better, and achieve faster convergence.
- With $E =20$, increasing C from 5 to 8 clients improved both the validation and test accuracy. It has also made the validation accuracy more stable.
- With $E =100$, increasing C from 5 to 8, has almost no impact on the model. It could be because these values are relatively close to one another. And with a high enough

epoch size, even using small number of clients can be sufficient to achieve good enough results. However, it seems that both $C=5$ and 8 are moderate and acceptable values. In further runs (see below), we notice that increasing C from 2 to 6 has positive impact of improving the accuracy of the model. This could be because $C=2$ is too small.

Comparisons at $T=100$

I tested several cases with only 100 communication rounds each. Ideally I wanted to test 3 variables for C , 3 variables for E , 3 variables for learning rate, and 3 variables of batch size for a total of 81 combinations. However due to limitation of time, I was only able to test half of those.

You can find the plots in the following folder: Part 1\Q1\Hyperparameter Tuning in one code. And the results that include the last training accuracy and last validation accuracy at round 100, along with test accuracy and test loss are included in the textfile: Results.txt in the same folder.

The runs that I was able to test include the following values:

$C_{\text{options}} = [2, 6]$

$E_{\text{options}} = [300, 60, 10]$

$LR_{\text{options}} = [0.00001, 0.0001, 0.01]$

$\text{batch_options} = [100, 50, 10]$

Observations:

1. $LR=0.01$ performed very poorly as compared to other values of LR when keeping the other hyperparameters constant and with the same number of clients. It barely showed any learning.
2. $LR=0.00001$ performs better with $\text{epochs}=300$ than 60 , and better with $\text{epoch}=60$ than 10 , where it barely learns in the latter. The validation accuracy almost triples when E increases from 10 to 60 , and also triples as it increases to $E=300$; and the trend is exponential as compared to linear in the case of $E=60$.
3. When comparing the batch size (for combination $C=2, E=300, LR=1e-05$), we notice that although the test accuracy is almost the same for all three combinations (with batch-size $=100, 50, 10$), and although the validation accuracy of the combination with batch size $=100$ is lower; the confidence and reliability of this value is higher for batch size $=100$ as the difference between validation accuracy and test accuracy is less than 0.1 (so 10%) , while it is higher than 0.2 (so 20%) for the other two cases. This could show overfitting issue with smaller batch sizes.

Although in another combination (with $C=2$, $E=60$, $LR=1e-05$, $batch=50$), batch size 50 has the highest validation accuracy, but the test accuracy is highest between these three combinations for batch size =10.

4. Increasing both E and C improves the validation and test accuracy if we compare the following three runs:

$C=6$, $E=300$, $LR=0.0001$, $batch=50$

Test Accuracy: 0.6484

train acc: 0.9920131653230241

validation acc: 0.775365451612371

test loss: 2.0628082752227783

test accuracy: 0.6484396457672119

$C=2$, $E=300$, $LR=0.0001$, $batch=50$

Test Accuracy: 0.5510

train acc: 0.9954888601954892

validation acc: 0.6783476263927899

test loss: 3.222238302230835

test accuracy: 0.5509527921676636

$C=2$, $E=10$, $LR=0.0001$, $batch=50$

Test Accuracy: 0.4424

train acc: 0.5480873191649197

validation acc: 0.6136594824264386

test loss: 2.3997039794921875

test accuracy: 0.44241923093795776

We notice that increasing E to 300 improved the validation and test accuracy within 100 rounds, and then increasing both E and C improved the validation and test accuracy even further.

5. $LR = 0.0001$ seem to improve training accuracy faster, and we notice a significant gap between the training and validation accuracy, while with $LR = 0.00001$, both training and validation accuracy are stuck around 0.7. This could show that $LR=0.00001$ is too small, and for our data is leading to local minima.
6. Based on the trends, increasing Epoch seem to always improve and speed up the learning, however it is computationally expensive.
7. Increasing the batch size has led to more memory issues, and warning messages about interrupted look of optimization within tensorflow (it tries again when interrupted).
8. Increasing the number of clients seem to reduce the fluctuation in validation and training accuracy (notice plots of $C=6$, $E=300$, $LR=1e-05$, $batch=10$, versus $C=2$, $E=300$, $LR=1e-05$, $batch=10$, as well as with different batch sizes.)

6. Q2 Parallel Clients Implementation

The code of this question is in folder Q2. Please note that ray could encounter several issues when there are not enough resources. Errors such as (RayActorError: The actor died unexpectedly before finishing this task.) could occur. In this case, one way to stop the error is to use only 1 CPU on each client instead of 1 CPU and 1 GPU.

```
@ray.remote(num_cpus=1, num_gpus=0)
```

This implementation speeds up the training significantly.

7. Q2 Parallel Clients Plots

Run- A

The following plots show the aggregated training and validation accuracy or running the training of the model on 4 parallel clients using Ray for RunA:

The hyperparameters selected after several runs and trial and error are the following:

- $lr = 0.0005$
- $batch_size = 8$
- $total_rounds = 500$
- $C = 4$
- $E = 40$

The above hyperparameters were selected as a tradeoff between adequate run-time, fast but steady convergence. And it is based on the lessons learned in the previous question.

500 communication rounds were enough to achieve convergence. (I added a convergence condition of a validation accuracy of 0.9 if achieved if the round is less than the upper bounds of communication bounds, but that can be changed to a for loop instead).

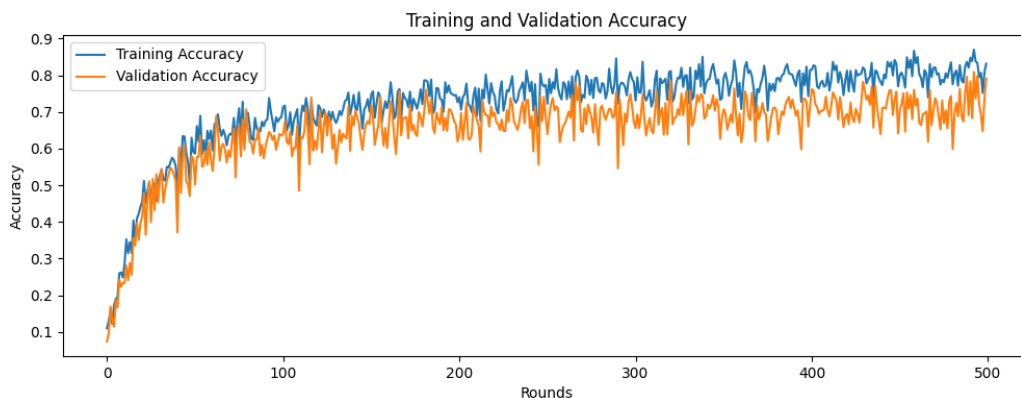


Figure 21: Training and Validation Accuracy for Q2-RunA model

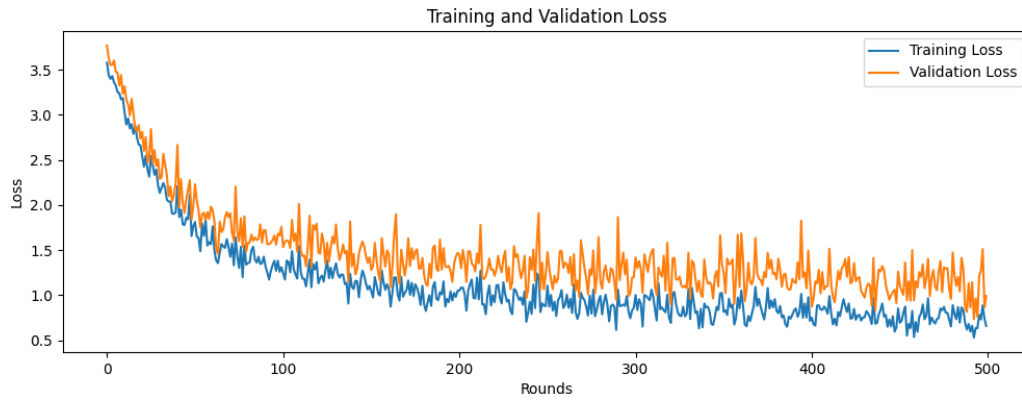
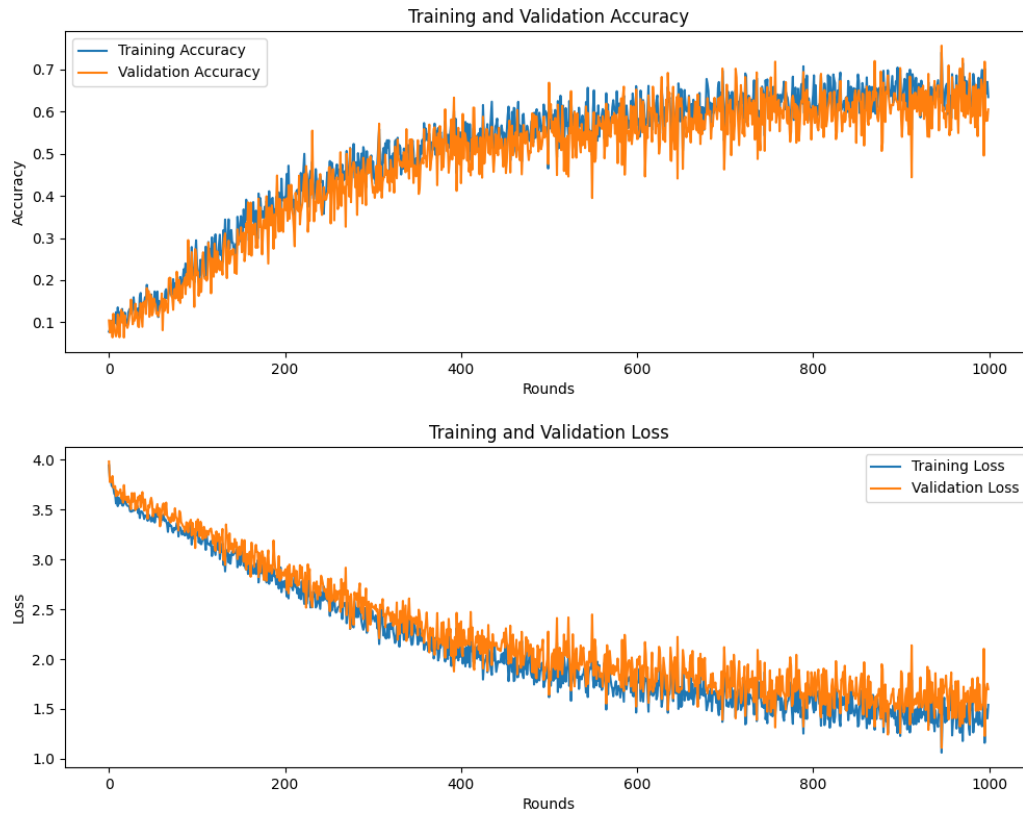


Figure 22: Training and Validation Loss for Q2-RunA model

Run B

- $lr = 0.0005$
- $batch_size = 8$
- $total_rounds = 1000$
- $C = 4$
- $E = 40$



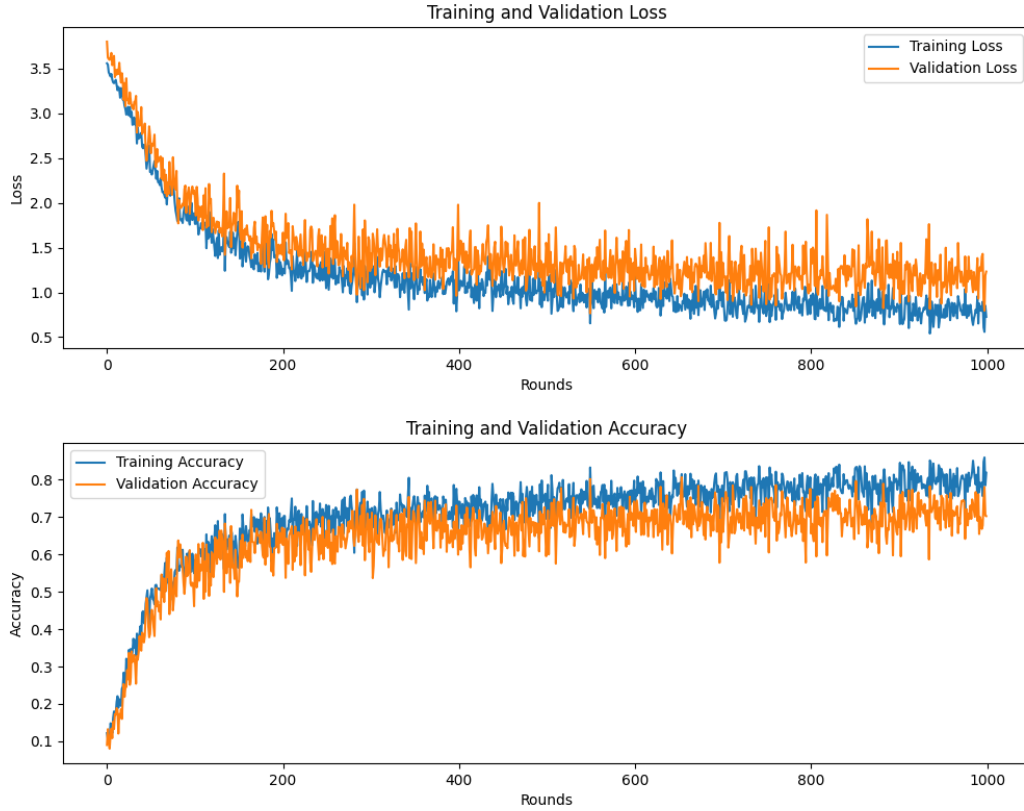
The plots of run B show convergence and learning, however, the learning is more steady than in run A and B and less fast.

Run C

- $lr = 0.0005$
- $batch_size = 16$
- $total_rounds = 1000$
- $C = 4$
- $E = 40$

Results:

- test loss: 1.40793776512146
- test accuracy: 0.6437448263168335



The plots in run C show exponential convergence and learning. However, given that we're only using 4 clients, having fluctuations in the accuracy and loss is inevitable.

8. Q2 Parallel Clients Test Accuracy

Testing the previous model in Q2-run A on the test data gives the following test metrics shown in the table below.

Table 1: Test data metrics using Q2-runA model

<i>Validation Accuracy (round 500)</i>	0.790421727074095
Test accuracy	0.644297182559967
Test Loss	1.3917763233184814